

Finding Loop Invariants and Creating Loops (continue)

1. Create a program (together with its full proof outline under total correctness) that calculates the function $sum(0, n)$, which sums up consecutive natural numbers from 0 to n and stores the result $s = sum(0, n)$.

- 1) First, Let's figure out the pre- and post- conditions from the question:

- To calculate $sum(0, n)$, n should be a named constant defined in the precondition: $n \geq 0$

- Postcondition is straightforward: $s = sum(0, n)$

This is what we get so far:

$$\{n \geq 0\} \dots \text{our program} \dots \{s = sum(0, n)\}$$

- 2) It looks like the program needs a loop to sum up numbers in a range. So secondly, let's try to find some possible loop invariants. Here, replacing a constant with a variable seems to be a good idea. There are two constants 0 and n in the postcondition, so we have two loop invariant candidates. We have already seen the loop created with a loop invariant where n is substituted by a variable; so here let's try to substitute n by a variable.

If we replace 0 by a variable k , we get $s = sum(k, n)$. Since we need the sum of the first n positive integers, and k will be equal to 0 after the loop, so k should be in the range from 0 to n : we can initialize $k = n$ and decrease it in each iteration until $k = 0$. And we will get a loop that looks like this:

```

{ $n \geq 0$ } ... some code ...
{inv  $s = sum(k, n) \wedge 0 \leq k \leq n$ } {bd  $k$ }
while  $k \neq 0$  do
    ... make  $k$  smaller and something else ...
od
{ $s = sum(k, n) \wedge 0 \leq k \leq n \wedge k = 0$ }          #  $p \wedge \neg B$ 
{ $s = sum(0, n)$ }
```

- When we replace a constant c / an expression e with a variable k , we can consider the range of k (if we have enough information). We usually end the program with $k = c$ (or the value of e); if we can figure out the initial value of k before the program starts (say d), then c (or the value of e) and d is the range of variable k .
- 3) Next, let's consider how to initialize the loop. The program starts with $n \geq 0$ and it currently doesn't imply the loop invariant, so we need to initialize some variables before the loop.

We are most likely starting the loop with $k = n$; which means we need $s = sum(k, n) \wedge 0 \leq k \leq n \wedge k = n$ before the first iteration, so $s = n$. Then:

```

{ $n \geq 0$ }  $k := n$ ; { $n \geq 0 \wedge k = n$ }  $s := n$ ;
{ $n \geq 0 \wedge k = n \wedge s = n$ }
{inv  $s = sum(k, n) \wedge 0 \leq k \leq n$ } {bd  $k$ }
while  $k \neq 0$  do
    ... make  $k$  smaller and something else ...
od
```

$$\{s = \text{sum}(k, n) \wedge 0 \leq k \leq n \wedge k = 0\}$$

$$\{s = \text{sum}(0, n)\}$$

- 4) Then, let's consider the loop-body. Other than updating the variable k , we also need $\{p \wedge B\} S \{p\}$ being valid.

k will be decreased by 1 after each iteration, and we need $s = \text{sum}(k, n)$ in the loop invariant. Thus, we can update $s := s + \text{sum}(k - 1, n) - \text{sum}(k, n)$. Then:

```

{ $n \geq 0$ }  $k := n$ ; { $n \geq 0 \wedge k = n$ }  $s := n$ ;
{ $n \geq 0 \wedge k = n \wedge s = n$ }
{inv  $s = \text{sum}(k, n) \wedge 0 \leq k \leq n$ } {bd  $k$ }
while  $k \neq 0$  do
   $s := s + k - 1$ ;  $k := k - 1$ 
od
{ $s = \text{sum}(k, n) \wedge 0 \leq k \leq n \wedge k = 0$ }
{ $s = \text{sum}(0, n)$ }

```

$p \wedge \neg B \Leftrightarrow q$
 $\neg B$

- The proof of the loop-body and some logic implications are omitted here.
- We can also try to create a loop invariant by adding some disjuncts or removing some conjuncts.
 - Adding disjuncts can be very open-ended; but since we need $p \wedge \neg B \Rightarrow q$, we can try for different loop condition B and let $p \equiv q \vee B$, then we have both $p \wedge \neg B \Rightarrow q$ and $q \Rightarrow p$. The loop will look like:

```

{inv  $q \vee B$ } {bd ...}
while  $B$  do
  { $(q \vee B) \wedge B$ }
  loop body
  { $q \vee B$ }
od
{ $(q \vee B) \wedge \neg B$ }
{ $q$ }

```

$(q \vee \underline{\quad}) \wedge \neg B \Leftrightarrow q$

- Removing conjuncts can be used when the postcondition is a conjunction $q \equiv q_1 \wedge q_2 \wedge \dots \wedge q_n$, where $n \geq 2$. It is natural to try to drop off some of q_k to get a loop invariant candidate: $p_k \equiv q_1 \wedge q_2 \wedge \dots \wedge q_{k-1} \wedge q_{k+1} \wedge \dots \wedge q_n$. Then the loop looks like:

```

{inv  $p_k$ } {bd ...}
while  $\neg q_k$  do
  { $p_k \wedge \neg q_k$ }
  loop body
  { $p_k$ }
od
{ $p_k \wedge q_k$ }
{ $q$ }

```

$p \wedge \neg B \Leftrightarrow q$

2. Create a program which can linear-search for x in an array slice $b[0 \dots n - 1]$ (note that, in our language, we don't have the expression $b[0 \dots n - 1]$ to represent the first n indices of b). The precondition is that array b has at least n elements ($n \geq 0$); x may or may not appear in $b[0 \dots n - 1]$. The postcondition should be k equals to the index of the leftmost occurrence of x in $b[0 \dots n - 1]$; if x is not found then let $k = n$.

- 1) Let us start with creating the pre- and post- conditions.

Precondition can be: $\text{size}(b) \geq n \wedge n \geq 0$

Let us define $x \notin b[0 \dots n - 1]$ with a predicate function $\text{NotIn}(x, b, n) \equiv \forall i. 0 \leq i < n \rightarrow x \neq b[i]$.

We notice that no matter whether x is in $b[0 \dots n - 1]$ or not, we always have $\text{NotIn}(x, b, k)$ if k is in returned index. Thus, postcondition can be written as:

$$\begin{aligned} & 0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge (k < n \rightarrow b[k] = x) \\ & \Leftrightarrow 0 \leq k \wedge k \leq n \wedge \text{NotIn}(x, b, k) \wedge (k < n \rightarrow b[k] = x) \end{aligned}$$

- 2) The postcondition is a conjunction, we can try to create a loop invariant by dropping some conjuncts. There are four conjuncts, and this means that we can try four candidates. Remember that, usually, the drop off conjunct will be considered as $\neg B$.

- a. If we drop off $0 \leq k$, then in the loop body we will have $k < 0$, and this is out of the bound of an array index. This is not a good idea.
- b. Similarly, if we drop of $k \leq n$, then in the loop body we will have $k > n$, which is not a guaranteed index in array b .
- c. Dropping off $\text{NotIn}(x, b, k)$ means the loop condition becomes $x \in b[0 \dots k - 1]$ (note that this is not a legal expression). The problem is how do we start this loop?
 - If we start with $k = 0$, then we are check whether x is in an array slice of length 0, which is fine; but then we need to check with $k = 1$, and we need $x = b[0]$, but it is not guaranteed.
 - If we start with $k = n$, then we need $x \in b[0 \dots n - 1]$, which is also not guaranteed. So, this is not a good idea.
- d. Dropping off $k < n \rightarrow b[k] = x$ could work. The loop condition will become $\neg(k < n \rightarrow b[k] = x) \Leftrightarrow k < n \wedge b[k] \neq x$. We can get a partial proof outline that looks like the following:

```

{size(b) ≥ 0 ∧ n ≥ 0} ... some code ...
{inv 0 ≤ k ≤ n ∧ NotIn(x, b, k)}                                     # p
{bd ...}
while k < n ∧ b[k] ≠ x do                                           # B
  {0 ≤ k ≤ n ∧ NotIn(x, b, k) ∧ k < n ∧ b[k] ≠ x}                 # p ∧ B
  loop body
  {0 ≤ k ≤ n ∧ NotIn(x, b, k)}                                       # p
od
{0 ≤ k ≤ n ∧ NotIn(x, b, k) ∧ (k < n → b[k] = x)}                 # p ∧ ¬B ⇔ q

```

- 3) We can start the loop with $k = 0$, so the before the loop we should have $0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge k = 0$. In each iteration, we simply increase k .

```

{size(b) ≥ 0 ∧ n ≥ 0} k := 0; {size(b) ≥ 0 ∧ n ≥ 0 ∧ k = 0}      # forward assignment
{inv 0 ≤ k ≤ n ∧ NotIn(x, b, k)}
{bd ...}
while k < n ∧ b[k] ≠ x do
  {0 ≤ k ≤ n ∧ NotIn(x, b, k) ∧ k < n ∧ b[k] ≠ x}

```

```

    { $0 \leq k + 1 \leq n \wedge \text{NotIn}(x, b, k + 1)$ } # backward assignment
     $k := k + 1$ 
    { $0 \leq k \leq n \wedge \text{NotIn}(x, b, k)$ }
  od
  { $0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge (k < n \rightarrow b[k] = x)$ }

```

- 4) $n - k$ is a good loop bound expression. Then we have a full proof outline of the program for linear search as follows:

```

{ $n \geq 0$ }  $k := 0$ ; { $n \geq k = 0$ }; { $\text{size}(b) \geq 0 \wedge n \geq 0 \wedge k = 0$ }
{inv  $0 \leq k \leq n \wedge \text{NotIn}(x, b, k)$ } {bd  $n - k$ }
while  $k < n \wedge b[k] \neq n$  do
  { $0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge k < n \wedge b[k] \neq n \wedge n - k = t_0$ }
  { $0 \leq k + 1 \leq n \wedge \text{NotIn}(x, b, k + 1) \wedge n - (k + 1) < t_0$ } # backward assignment
   $k := k + 1$ 
  { $0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge n - k < t_0$ }
od
{ $0 \leq k \leq n \wedge \text{NotIn}(x, b, k) \wedge (k < n \rightarrow b[k] = n)$ }

```

3. Given two non-empty sorted arrays b_1 and b_2 , find the least indices i and j such that $b_1[i] = b_2[j]$; if no such i and j exist, end with $i = n$ or $j = m$ such that $n = \text{size}(b_1)$ and $m = \text{size}(b_2)$.

- We have seen the three-array version of this problem when we introduce nondeterministic statements: our algorithm starts with $i = j = 0$ then increase either i or j in each iteration. This time we focus on the loop invariant and termination.

- The precondition of the program can be: $\text{size}(b_1) = n > 0 \wedge \text{size}(b_2) = m > 0 \wedge \text{Sorted}(b_1) \wedge \text{Sorted}(b_2)$, where $\text{Sorted}(b) \equiv \forall 0 \leq k < \text{size}(b) - 1. b[k] \leq b[k + 1]$. Since $\text{Sorted}(b_1) \wedge \text{Sorted}(b_2)$ is always true during the program and our searching procedure doesn't change it; we will only show it in the precondition and omit it everywhere else.
- Our algorithm will only return the left-most match since the search starts from index 0. So, if the program ends with $b_1[i]$ and $b_2[j]$, then there is no match on the left of i and j . This is similar to the postcondition of the linear search, we can write postcondition $q \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j) \wedge (i < n \wedge j < m \rightarrow b_1[i] = b_2[j])$, where $\text{NoMatch}(b_1, b_2, i, j) \equiv \forall 0 \leq i' < i. \forall 0 \leq j' < j. b_1[i'] \neq b_2[j']$.
- Like linear search, we can also get invariant by dropping of the last conjunct, and $p \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j)$, then we have loop condition $B \equiv \neg(i < n \wedge j < m \rightarrow b_1[i] = b_2[j]) \Leftrightarrow i < n \wedge j < m \wedge b_1[i] \neq b_2[j]$.
- Since we are increasing i or j in each iteration, we define bound function $t(i, j) \equiv (n - i) + (m - j)$. It is easy to see that $t(i, j) \geq 0$. Then we can get the following partial program:

```

{ $\text{size}(b_1) = n > 0 \wedge \text{size}(b_2) = m > 0 \wedge \text{Sorted}(b_1) \wedge \text{Sorted}(b_2)$ }
{ $0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j)$ }
 $i := 0; j := 0;$ 
{inv  $p \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j)$ } {bd  $t(i, j) \equiv (n - i) + (m - j)$ }

```

```

while  $B \equiv i < n \wedge j < m \wedge b_1[i] \neq b_2[j]$  do
     $\{p \wedge B \wedge t(i, j) = t_0\}$ 
    ...increase  $i$  or  $j$ , and maybe something else ...
     $\{p \wedge t(i, j) < t_0\}$ 
od
 $\{p \wedge (i < n \wedge j < m \rightarrow b_1[i] = b_2[j])\}$ 
 $\{q \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j) \wedge (i < n \wedge j < m \rightarrow b_1[i] = b_2[j])\}$ 

```

- What program should be the loop body? Let's use a deterministic program this time. We want to increase $i := i + 1$ when $b_1[i] < b_2[j]$, since $b_2[j]$ is already too large so increasing j won't help; symmetrically, we want to increase $j := j + 1$ when $b_1[i] > b_2[j]$. With a conditional statement, we have the following partial proof outline:

```

 $\{size(b_1) = n > 0 \wedge size(b_2) = m > 0 \wedge \text{Sorted}(b_1) \wedge \text{Sorted}(b_2)\}$ 
 $\{0 \leq 0 \leq n \wedge 0 \leq 0 \leq m \wedge \text{NoMatch}(b_1, b_2, 0, 0)\}$ 
 $i := 0; j := 0;$ 
 $\{\text{inv } p \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j)\} \{\text{bd } t(i, j) \equiv (n - i) + (m - j)\}$ 
while  $B \equiv i < n \wedge j < m \wedge b_1[i] \neq b_2[j]$  do
     $\{p \wedge B \wedge t(i, j) = t_0\}$ 
    if  $b_1[i] < b_2[j]$  then # Conditional Rule 1
         $\{p \wedge B \wedge t(i, j) = t_0 \wedge b_1[i] < b_2[j]\} i := i + 1 \{p \wedge t(i, j) < t_0\}$ 
    else
         $\{p \wedge B \wedge t(i, j) = t_0 \wedge b_1[i] > b_2[j]\} j := j + 1 \{p \wedge t(i, j) < t_0\}$ 
    fi
     $\{p \wedge t(i, j) < t_0\}$ 
od
 $\{p \wedge (i < n \wedge j < m \rightarrow b_1[i] = b_2[j])\}$ 
 $\{q \equiv 0 \leq i \leq n \wedge 0 \leq j \leq m \wedge \text{NoMatch}(b_1, b_2, i, j) \wedge (i < n \wedge j < m \rightarrow b_1[i] = b_2[j])\}$ 

```